

This note tells you what to expect when you host a Prolific study built from this template on Railway, and it is backed by measurements rather than guesses. A study forked from this template ( `exp_pilots` ) ran its real data collection on Railway and we now have its telemetry under genuine participant load. The numbers below are all from that run. For the step-by-step how-to — provisioning the container, the managed database, deploy commands — see the companion, `docs/hosting_a_prolific_study.md` ; this document is the “does it cope, and how big does it need to be” answer that sits alongside it.

This supersedes the earlier `_ai/railway_report.tex` (the 2026-08-23 ten-place timing wave), which observed a peak of only 9 concurrent participants and *projected* resource use linearly out to 220. That projection is broadly confirmed here by a real run: it forecast roughly 0.39 vCPU and 0.52 GB at 220 concurrent, and the measured run below drew 0.26 vCPU and 0.41 GB at 81 concurrent — the same small envelope, now measured rather than extrapolated.

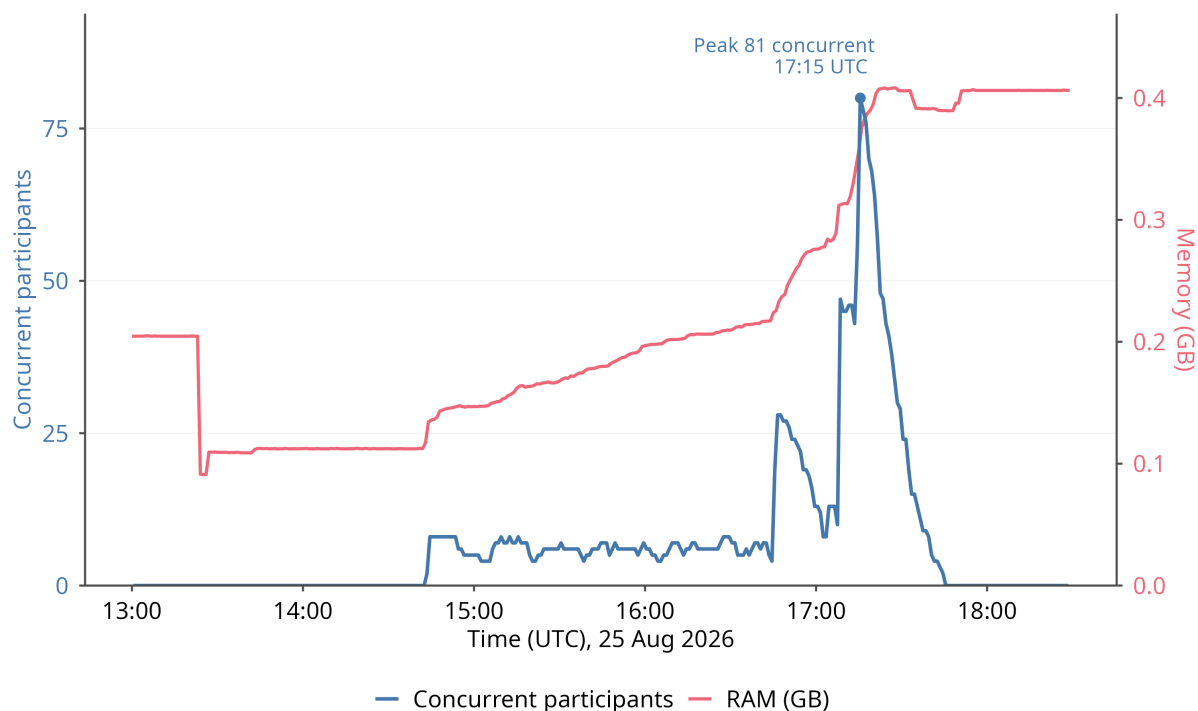
## The stack

The whole deployment is **one small Railway container running the oTree app, plus Railway's managed Postgres**. There is no load balancer, no worker pool, no cache tier, no autoscaling group. This is the setup the template is designed for, and the measurements below are its verdict: for a Prolific-scale study it is not merely adequate, it has large headroom.

## The load that was served

The `exp_pilots` pilot drew **195 participants** in total, across two Prolific waves: a small release (15 people, never more than 9 in the task at once) and the main batch of **180**. Sessions were short — a **median of 12.3 minutes** — and that is exactly what makes the load *bursty* rather than sustained: Prolific opens the places, people arrive in a clump, move through the task quickly, and drain away.

The main batch is the real stress test. Arrivals trickled in under ten concurrent through the afternoon, then the Prolific batch release produced a sharp ramp that peaked at **81 simultaneous participants at 17:15 UTC** before draining by roughly 17:40. That single hour carried **9,656 requests**, out of **31,390** across the whole run (Aug 14 – Sep 12); everything outside the two waves was near-idle traffic. Figure 1 shows the concurrency burst against the container's memory over the same window.



**Figure 1:** Concurrent participants (left axis) and container RAM (right axis) over the Aug 25 wave. The burst peaks at 81 concurrent at 17:15 UTC; memory rises with it to 0.41 GB.

## What it cost the box

The striking result is how cheap 81 concurrent participants turned out to be.

- **CPU** was essentially nothing when idle (about 0.001–0.01 of one core). At the burst it rose to a peak of **0.26 cores — 26% of a single core — and only for about a minute**; averaged over the whole peak hour, CPU use was around **6%**.
- **RAM** ran at roughly **0.1–0.3 GB** through the day and peaked at just **0.41 GB** during the burst.

So the worst moment of the run consumed about **a quarter of one CPU core and under half a gigabyte of memory**. That is a small container barely being touched, not one under strain. The managed Postgres absorbed every participant write without appearing anywhere in the container's resource picture.

## Latency: did it slow down under load?

No — if anything it was faster when busy. Comparing the quiet warmup hour with the peak hour:

- **Warmup (13:00, near-idle):** p50 96 ms, p90 131 ms, p99 178 ms.
- **Peak (17:00, 9,656 requests):** p50 25 ms, p90 101 ms, p99 406 ms.

The median response was *lower* at the peak than during the quiet hour — a warm container with hot caches under real traffic beats a cold idle one — and even the 99th-percentile tail stayed under half a second. There is no sign of queueing, thrashing, or degradation as concurrency climbed.

## Reliability

Server errors were vanishingly rare and **none touched a participant**. Across the whole run there were **26 5xx responses — 0.08% of all requests** — and every one fell inside two deploy-crash windows during near-idle traffic: a two-step deploy booted against the old database (Aug 17, 3 × 502), and a deploy that reported success while the container had actually exited (Aug 22, 23 × 502/500). Both were operator mistakes during quiet windows, not platform failures under load, and there were **zero 5xx errors during either participant wave**. The remaining non-5xx noise is what any public URL attracts: 1,255 404s from bots and scanners, and 910 499s from clients closing the connection early.

## Cost

Railway bills **per minute of uptime**, so the true cost of a study is only the hours the service is actually running — not the calendar period it spans. At the observed average resources (~0.43 GB RAM, ~0.005 of a vCPU, at ~\$10/GB-month and ~\$20/vCPU-month) the app costs about **\$0.006 per hour of uptime**, or roughly **\$0.01/hour all-in with the managed Postgres**. The actual study — a few hours of real serving across the two waves — therefore cost on the order of **a few cents**, and even generously leaving the service up for a whole day per wave stays well under a dollar.

The ~\$2–3 we actually paid was not the cost of the study: it was the cost of **leaving the service up, idle, for ~18 days**. Over that window the project used **11,109 GB-minutes of RAM** and **120.6 vCPU-minutes** — about \$2.6 of memory and \$0.06 of CPU — which still sits inside the \$5/month Hobby plan's included usage, but is mostly idle time we did not need to pay for. The practical guidance follows directly: **spin the service up shortly before recruiting and tear it down (or pause it) once the data is exported**, keeping the managed Postgres or a snapshot if you need to serve returns or reviews later.

The prior setup, Heroku, bills the opposite way: a **fixed monthly amount regardless of use**. Paid dynos start at ~\$7 (Basic), ~\$25 (Standard-1X) or ~\$50 (2X) each with no free tier since 2022, plus a Postgres add-on (~\$5–50/month), plus historically an oTree Hub subscription — so a few dynos + Postgres + Hub is on the order of **tens of dollars per month, whether the study runs for three hours or not at all**. For a short study the gap is not an order of magnitude, it is dramatic: **cents of actual-uptime compute on Railway versus tens of dollars of fixed monthly cost on Heroku** — and Railway is a single dashboard rather than a multi-piece subscription stack.

## Scaling verdict

A single small Railway container backed by managed Postgres served the entire pilot without strain: at the busiest moment, 81 participants in the task at once cost about 26% of one CPU core and 0.41 GB of RAM, response times held at a 25 ms median (faster under load than at rest), and not one server error reached a participant. With CPU peaking near a quarter of one core and memory under half a gigabyte, this setup has room to absorb **several multiples of 81 concurrent** before the container itself would need to grow, and the managed Postgres scales independently of it. Host your study on the same single-container-plus-managed-Postgres setup with no architectural change; the practical lever, if a future release ever bunches arrivals harder than this one did, is **Prolific's batch-release pacing, not a bigger server**. The one operational rule this run teaches — because the only failures in it were self-inflicted deploys — is **never deploy a schema change while participants are live**: guard it with the RESET\_DB-before-schema-deploy rule, `scripts/verify_deploy.py`, and the `/health` healthcheck so a dead container can no longer report a successful deploy.